A Routing-Aware Federated Learning Framework for Secure and Resilient Edge–Cloud IoT Intrusion Detection

Nahideh Derakhshanfard[1], Reza Azarnia[1], Amir Mollanejad[2], Ali Asghar Pour Haji Kazem[3], Neda Dadashkhani[4], Parisa Khoshvaght[5], Mehdi Hosseinzadeh[6]

[1]*Department of Computer Engineering, Tabriz Branch, Islamic Azad University, Tabriz, Iran*
[2]*Department of Computer Engineering, Aras Branch, Islamic Azad University, Jolfa, Iran*
[3]*Department of Software Engineering, Faculty of Engineering and Natural Sciences, Istinye University, Istanbul, Turkey*
[4]*Computer Programming Program, Vocational School, Istinye University, Istanbul, Turkey*
[5]*Institute of Research and Development, Duy Tan University, Da Nang, Vietnam*
[6]*School of Engineering & Technology, Duy Tan University, Da Nang, Vietnam*

Co-author: n.derakhshan@iaut.ac.ir

Abstract

In federated intrusion detection, unreliable update delivery and abnormal client updates can both affect model training. We evaluate a routing-aware Edge–Cloud federated learning framework combining path and client selection, median-based surrogate imputation, MAD filtering, clipping, capped weighting, optional server-side Gaussian perturbation, and momentum. Controlled experiments use CICIoT2023 and UNSW-NB15, five paired seeds (12–16), and the report partition of a locked validation split; the reserved test sets were not accessed. On CICIoT2023, the proposed configuration with diagnostic $\widehat{\varepsilon} \approx 20$ achieves F1=0.9914 and ROC–AUC=0.9937. Relative to FedAvg with uniform selection and random paths, its mean delivery delay decreases from 87.3 to 66.0 ms, while first-transmission PDR increases from 0.694 to 0.851. On UNSW-NB15, the same proposed configuration attains F1=0.9065 and ROC–AUC=0.9522; a controlled MAD-filter comparison improves ROC–AUC by 0.098 over FedAvg with selection and paths held fixed. Independent path-assignment and client-selection controls improve simulated network metrics without a detectable change in selected-checkpoint ROC–AUC. The complete privacy pipeline decreases ROC–AUC by 0.0032 and 0.0419 on the two datasets, respectively. Five-seed effects remain uncertain, and the diagnostic privacy accounting does not establish an end-to-end differential-privacy guarantee.

Keywords: Federated learning, Internet of Things, Edge–Cloud architecture, Intrusion detection, Routing-aware aggregation, Fault tolerance, Privacy-aware aggregation, Network resilience

1. Introduction

The rapid expansion of Internet of Things (IoT) systems in smart cities, digital healthcare, intelligent manufacturing, transportation, and critical infrastructure has generated massive volumes of distributed, heterogeneous, and highly sensitive data. Under such conditions, constant transfer of raw data from IoT sensors to the cloud increases communication overhead and latency while simultaneously increasing the risk of information leakage, violation of user privacy, and cyberattacks. Edge–Cloud architectures have therefore emerged as a practical approach for reducing latency, distributing computational load, and improving scalability. However, the dynamic nature of IoT networks, the limited resources of edge devices, data heterogeneity, link instability, and evolving security threats still hinder the development of stable and reliable intrusion detection systems (IDSs).

In this context, federated learning (FL) provides a distributed learning paradigm that enables IDS models to be trained without transferring raw data, thereby reducing the risk of information exposure. SecureDyn-FL shows that privacy-preserving and robustness-oriented mechanisms can strengthen the reliability of FL-based IDS models in IoT networks [1]. Complementary work on federated and incremental IDSs further suggests that such models should remain accurate while adapting to emerging attack patterns and gradual shifts in network traffic [2]. Multi-model and hierarchical federated learning techniques have also been introduced to improve intrusion detection in heterogeneous IoT systems [3].

Despite these advantages, keeping data at the source is not sufficient by itself to ensure security and privacy. Model updates, gradients, or transmitted parameters may still reveal sensitive information about local data through inference attacks. Moreover, in Edge–Cloud environments, clients differ in computational capacity, connectivity, availability, and trustworthiness, which can affect the convergence of the global model. S2D-FL demonstrates that sparsity-guided and trust-aware strategies can improve robustness, communication efficiency, and privacy preservation in IoT edge federated learning [4]. Furthermore, trustworthy FL frameworks that jointly consider privacy and Byzantine resilience indicate that FL security should go beyond decentralized model training and include client behavior analysis, reliability assessment, and robust aggregation [5].

One of the major issues in FL-based IoT systems is the

presence of malicious, faulty, or untrustworthy clients. Such clients may transmit updates that deviate from the intended optimization direction because of poisoning attacks, gradient manipulation, hardware failures, or Byzantine behavior. Secure and Byzantine-robust aggregation methods have been developed to limit the influence of abnormal updates, including approaches based on anonymous participation and secure aggregation [6]. However, effective defenses must distinguish malicious behavior from legitimate differences caused by non-IID data [7]. Convergent aggregation and ensemble-based mechanisms further address poisoning and compromised local models [8], while recent work highlights the risk of excluding useful clients through excessively aggressive filtering [9].

The privacy-preserving exchange of model updates is another key concern in FL-based IoT systems. Methods such as TriSAFE combine transcript-bound verifiable secure aggregation, differential privacy, and timing defenses, showing that FL security in gateway-assisted IoT environments should consider both model exchange and communication timing [10]. Decentralized secure aggregation solutions for zero-trust device-to-device networks indicate that complete dependence on a central server and static trust assumptions among nodes are not suitable for open and dynamic environments [11]. Although these approaches can provide strong confidentiality and secure aggregation, they often introduce considerable computational or communication overhead. Moreover, most of these approaches do not directly address path selection, end-to-end delay, link failures, or path stability in Edge–Cloud IoT environments.

Beyond security and privacy, communication reliability and fault management are also critical for FL in IoT. In industrial and edge environments, data are often clustered, heterogeneous, and unstable, while client participation may be interrupted by link failures, congestion, or energy constraints. FedCST shows that FL reliability in industrial IoT depends on managing clustered data distributions and maintaining training stability [12]. However, communication in IoT is not limited to client selection; the transmission path of model updates also affects latency, packet delivery ratio, communication security, and training stability.

Recent work on federated edge intelligence has shown that combining FL with decision-making models, such as gated recurrent units and reinforcement learning, can support secure and adaptive routing in IoT networks [13]. Federated adaptive routing protocols for large-scale IoT systems show that jointly considering energy consumption, latency, and path quality can improve network communication efficiency [14]. Zero-trust and load-aware federated scheduling approaches, such as EdgeGuard-AI, further show that latency reduction in IoT edge networks requires joint consideration of security, network load, trust, and node communication status [15].

Despite these advances, most existing studies address only part of the overall problem. Some works mainly focus on intrusion detection and IDS model quality, but they do not explicitly incorporate path stability or communication failure into the learning process. Other studies emphasize privacy preservation, secure aggregation, or Byzantine fault tolerance while ignoring the impact of unstable paths, network latency, and failed update delivery on FL decision-making. Conversely, edge routing and scheduling approaches often aim to reduce latency or energy consumption, but they rarely integrate robust filtering for suspicious updates, central Gaussian privacy noise, and failure recovery within a unified FL framework.

Therefore, an important research gap remains: the lack of a unified framework that jointly addresses intrusion detection, network-aware path selection, privacy preservation, robust filtering, and fault tolerance in Edge–Cloud-based IoT systems. To address this gap, this paper proposes a routing-aware, privacy-aware, and fault-tolerant FL framework for intrusion detection in Edge–Cloud IoT systems.

In the proposed framework, IoT nodes train local models without sharing raw data and transmit only model updates through selected paths to the Edge–Cloud server. Latency, path stability, and security indicators guide client and route selection so that the FL process adapts to the current network state. To reduce the impact of client and link failures, failed updates are reconstructed using coordinate-wise median reconstruction when sufficient valid updates are available. Then, median absolute deviation (MAD)-based robust filtering around a robust center is applied to control suspicious updates. Finally, reliable updates are aggregated using clipping, capped weighting, and central Gaussian noise, and the global model is updated using server momentum. The proposed mechanism provides an estimated central differential privacy budget for the released aggregated update and global model under a trusted or semi-trusted server assumption.

The main contributions of this paper are summarized as follows:

- An integrated FL framework is proposed to jointly address intrusion detection, routing-aware communication, privacy-aware aggregation, and fault tolerance in Edge–Cloud IoT systems.
- A routing-aware client and path selection scheme is designed using latency, path stability, and route-security indicators to reduce end-to-end delay and improve communication reliability.
- A failure-recovery mechanism is introduced using coordinate-wise median reconstruction and reduced weighting to retain failed-client contributions in a controlled manner.
- A MAD-based robust filtering mechanism is applied to suppress unreliable, suspicious, or highly deviating updates before aggregation.
- Clipping, capped weighting, central Gaussian noise, and server momentum are combined to balance estimated privacy, learning stability, and IDS performance.

The remainder of this paper is organized as follows. Section 2 reviews related work on FL-based techniques for IoT security, privacy preservation, resilience, and routing. Section 3 describes the proposed framework, system architecture, mathematical formulation, and algorithmic procedure. Section 4 presents the simulation environment, datasets, evaluation metrics, and experimental results. Finally, Section 5 concludes the paper and outlines future research directions.

2. Related Work

This section reviews federated-learning-based studies related to IoT security, fault tolerance, privacy preservation, and routing optimization in Edge–Cloud IoT systems. The reviewed studies are organized into four main categories: federated learning for security and intrusion detection, fault-tolerant and resilient federated learning, privacy and data-security approaches, and routing and network-efficiency optimization. These research directions are reviewed to identify the remaining gaps addressed by the proposed framework.

2.1. Federated Learning for Security and Intrusion Detection in IoT

Recent federated learning research has increasingly addressed IoT network security because resource-constrained and open IoT environments are vulnerable to denial-of-service attacks, malicious data injection, model manipulation, and other cyberattacks. Almeida et al. proposed a modular and resilient FL framework for dynamic edge environments [16]. Their modular design supports model adaptation when devices generate data at different rates and operate under limited computational resources.

Aburukba proposed an FL-driven request-scheduling approach for IoT services in cloud data centers [17]. The results showed that FL-based scheduling can improve service quality and fault tolerance under variable workloads.

Several studies have focused more directly on intrusion detection. A performance-evaluation and data-scaling study of FL-based IDSs showed that data scalability, model selection, and aggregation behavior strongly affect detection accuracy in IoT networks [18]. Similarly, an FL-based denial-of-service detection method using ensemble feature selection and deep learning improved attack detection by focusing on more discriminative traffic features [19]. However, the additional feature-selection stage may increase computational cost.

Fault-tolerant and privacy-preserving IDS designs have also been investigated. Chen et al. introduced a multicenter hierarchical FL architecture with fault-tolerance mechanisms to improve resilience in edge computing networks [20]. Geng et al. developed a privacy-preserving FL-based IDS framework using secure multi-party computation [21]. In addition, Bhardwaj et al. investigated FL-based routing optimization in opportunistic IoT networks, showing that FL can also contribute to routing-related decision-making [22].

Overall, these studies demonstrate the relevance of federated learning for IoT security, intrusion detection, resilience, and routing support. However, most of them primarily focus on detection accuracy, scheduling, or isolated robustness mechanisms. The joint integration of intrusion detection, routing-aware communication, privacy-aware aggregation, and failure recovery therefore remains limited.

2.2. Fault-Tolerant and Resilient Federated Learning in Edge Networks

Fault tolerance and resilience are essential requirements in IoT and edge networks, where node failures, unstable links, bandwidth fluctuations, and intermittent participation can disrupt the federated learning process.

Haripriya et al. investigated fault-tolerance and latency optimization using edge servers and pre-trained models [23]. Their results suggest that distributing computation between edge and cloud resources can reduce latency and decrease dependence on a central server.

Liu et al. proposed a secure federated learning method with configurable reliability for heterogeneous edge environments [24]. This method allows different reliability levels to be assigned according to node conditions and resource availability.

Adaptive federated learning methods have also been studied for resource-constrained IoT devices. Mughal et al. examined adaptive federated learning through edge intelligence and multi-edge clustering [25]. Their results show that dynamic adaptation can help reduce latency and improve model quality under resource-constrained conditions.

Liu et al. proposed a federated-learning-oriented edge computing framework for the Industrial Internet of Things [26]. In industrial environments, resilience is particularly important because communication may be affected by disconnections, interference, and changing operational conditions.

Zhou et al. showed that federated learning can improve quality of service in Edge–Cloud architectures [27]. Lan et al. proposed a secure adaptive federated learning method based on the Cheon–Kim–Kim–Song (CKKS) encryption scheme [28]. Although this method improves data security, it also introduces additional cryptographic and computational overhead.

Taken together, these studies show that resilient federated learning in Edge–Cloud environments requires adaptive participation, low-latency communication, and mechanisms for handling unreliable clients or communication links. Nevertheless, most existing fault-tolerant federated learning studies do not explicitly combine failure recovery with routing-aware path selection, robust update filtering, and privacy-aware aggregation in a unified intrusion-detection framework.

2.3. Privacy and Data Security Approaches in Heterogeneous Environments

Although federated learning reduces the need to transmit raw data, it does not automatically eliminate privacy risks. Model updates, gradients, and shared parameters may still disclose sensitive information through reconstruction or inference attacks. Therefore, several studies have investigated privacy-preserving and data-security mechanisms for heterogeneous federated learning environments.

Peng et al. proposed an FL-based intrusion detection system using knowledge distillation for non-IID IoT environments [29]. Their method aims to improve model performance under heterogeneous data distributions by transferring knowledge between local and shared models. Although knowledge distillation may help local models benefit from shared knowledge, it can also introduce additional training complexity.

Trust management and decentralized security have also received considerable attention. Wang et al. introduced a blockchain- and federated-learning-based trust-management system for IoT

devices [30]. In this approach, blockchain provides an immutable record of device behavior and trust-related information.

Rahmati and Pagano proposed an FL-driven cybersecurity framework for IoT networks with privacy-preserving and real-time threat-detection capabilities [31]. Li et al. developed an edge privacy-preserving federated learning scheme using lightweight cryptographic techniques [32]. Such lightweight mechanisms are particularly relevant to resource-constrained IoT devices, where conventional cryptographic solutions may impose high computational and communication costs.

Bayat et al. combined Q-learning and blockchain for secure IoT data sharing at the edge [33]. Albogami proposed a deep federated learning security model for IoT-enabled edge computing environments [34]. Padmavathi and Saminathan introduced a trust-based access-control framework for secure and privacy-preserving IoT systems [35].

These studies show that privacy-aware federated learning often requires additional mechanisms such as cryptography, trust management, blockchain, access control, or knowledge distillation. However, these mechanisms may increase computational or communication overhead and are not always designed together with routing stability, failed-update recovery, or robust aggregation.

Therefore, privacy preservation in Edge–Cloud IoT systems should be considered together with network reliability, fault tolerance, and update-level robustness rather than as an isolated objective.

2.4. Routing and Network-Efficiency Optimization in FL-Based Edge–Cloud Architectures

Routing in IoT networks is challenging because devices are heterogeneous, bandwidth is limited, communication links may be unstable, and transmission latency can directly affect federated learning performance. Consequently, several studies have investigated routing optimization, load management, and network-efficiency improvement in federated edge and Edge–Cloud systems.

Bhardwaj et al. specifically investigated opportunistic routing in IoT networks and showed that federated learning can support routing decisions without requiring the direct exchange of raw data [22]. Such an approach can improve adaptability under congestion, communication fluctuations, and changing network conditions.

Although some studies do not directly focus on routing, they provide mechanisms closely related to communication efficiency. For example, modular federated learning, request scheduling, Industrial Internet of Things edge computing, and quality-of-service-oriented federated learning improve Edge–Cloud coordination, load distribution, service quality, and scheduling efficiency [16, 17, 26, 27]. These mechanisms provide important foundations for reliable and efficient routing in Edge–Cloud systems.

However, routing-oriented studies commonly focus on latency, energy consumption, or path optimization, while security, privacy, robust filtering, and failure recovery are considered separately or are not integrated into the same framework.

Taken together, the reviewed studies establish the importance of federated learning for IoT security, resilience, privacy, and communication efficiency. Related approaches to edge coordination, request scheduling, and quality-of-service management address important network requirements [16, 17, 26, 27]. However, intrusion detection, routing-aware selection, robust aggregation, and failed-update recovery are generally examined as separate concerns. This motivates the integrated Edge–Cloud framework investigated in this study. Table 1 summarizes the main characteristics, strengths, and limitations of the reviewed approaches.

Table 1: Summary and comparison of related work.

Ref./year	Method	Main focus	Main strength	Main limitation
[16], 2025	Modular edge FL	Dynamic edge resilience	Scalable and resilient	Limited scaling evaluation
[17], 2025	FL request scheduling	Fault-tolerant service scheduling	Higher fault tolerance	Timing-sensitive
[18], 2025	FL-based IDS study	Intrusion-detection evaluation	Accurate IDS performance	No routing mechanism
[19], 2025	FL with ensemble selection	DoS attack detection	High detection accuracy	High computational cost
[20], 2025	Hierarchical FL	Fault-tolerant edge learning	High learning stability	Architectural complexity
[21], 2024	SMPC-based FL	Privacy-preserving aggregation	Strong privacy protection	Heavy cryptographic cost
[22], 2024	FL-based routing	Opportunistic IoT routing	Improved routing decisions	Limited-scale evaluation
[23], 2025	Edge-assisted FL	Fault tolerance and latency	Lower communication overhead	Model-dependent
[24], 2026	Reliable FL	Configurable reliability	Effective reliability control	Performance loss under heterogeneity
[25], 2024	Adaptive FL	Resource-constrained IoT	Suitable for IoT devices	Possible accuracy loss
[26], 2024	Industrial IoT FL	Resilient edge computing	Improved resilience	No explicit security mechanism
[27], 2024	QoS-oriented FL	Edge–Cloud quality of service	QoS improvement	No attack-tolerance mechanism
[28], 2025	CKKS-based FL	Encrypted federated learning	Strong data security	High cryptographic overhead
[29], 2025	Knowledge-distillation FL	IDS under non-IID data	Effective under heterogeneity	Additional training cost
[30], 2025	Blockchain-based FL	IoT trust management	Strong trust management	Blockchain-induced delay
[31], 2025	Cybersecurity FL	Privacy and real-time detection	Real-time threat detection	Possible accuracy degradation
[32], 2023	Edge privacy FL	Lightweight privacy protection	Lightweight and secure	Limited application scope
[33], 2025	RL and blockchain FL	Secure IoT data sharing	Dynamic and reliable	High reinforcement-learning cost
[34], 2025	Deep FL security	IoT edge security	Enhanced security	High computational demand
[35], 2025	Trust-based edge FL	Access control and privacy	Strong trust-based access	Sensitive to trust estimation
Proposed, 2026	Routing-aware FL	IDS, routing, privacy, and recovery	Unified Edge–Cloud FL framework	Simulation-based; trusted or semi-trusted server

3. Proposed Method

This study proposes a federated learning framework with routing-aware, privacy-aware, and fault-tolerant mechanisms for intrusion detection in Edge–Cloud IoT systems. The proposed method aligns the federated learning process with the dynamic state of the underlying communication network.

Client and communication-path selection are therefore not determined solely by training participation. They also consider network-level indicators, including end-to-end delay, path stability, and communication security. In this study, the Edge–Cloud server refers to the orchestration and aggregation component that receives model updates through selected edge communication paths and performs recovery, filtering, privacy-aware aggregation, and global-model updating.

As illustrated in Figure 1, the framework operates through four stages in each communication round. First, clients and communication paths are selected using network-delay, path-stability, and security-related indicators. Second, selected clients train their models locally and transmit model updates without sharing raw data. Third, the server identifies missing or abnormal updates and applies robust filtering and surrogate recovery, with the processing order depending on the evaluated configuration. Finally, retained updates undergo clipping and capped weighted aggregation, followed by optional server-side Gaussian perturbation and a momentum-based global-model update.

3.1. Routing-Aware Client and Path Selection

At the beginning of each communication round, before model updates are exchanged, the proposed framework evaluates the communication state of the Edge–Cloud network. In IoT environments, the quality of the transmission path directly affects end-to-end delay, packet delivery ratio, communication stability, and the reliability of the federated learning process.

Therefore, the proposed method does not select communication paths solely based on link availability. Instead, delay, path stability, and path security are jointly considered in the routing decision. For each available client i in communication round t, a set of candidate paths $\mathcal{P}_i^{(t)}$ is considered. The routing score and the best selected path are defined as

$$\begin{aligned} R_{i,p}^{(t)} &= \lambda_L \widehat{L}_{i,p}^{(t)} + \lambda_S \widehat{S}_{i,p}^{(t)} + \lambda_Q \widehat{Q}_{i,p}^{(t)}, \\ p_i^{*(t)} &= \arg\max_{p \in \mathcal{P}_i^{(t)}} R_{i,p}^{(t)}. \end{aligned} \tag{1}$$

In Eq. (1), $\widehat{L}_{i,p}^{(t)}$ denotes the normalized delay score, where a larger value represents lower delay and better communication quality. Similarly, $\widehat{S}_{i,p}^{(t)}$ and $\widehat{Q}_{i,p}^{(t)}$ denote the normalized path-stability and path-security scores, respectively. The non-negative weighting coefficients satisfy $\lambda_L + \lambda_S + \lambda_Q = 1$.

The participating clients are selected according to the best achievable routing score of each client and the client participation ratio ρ:

$$\begin{aligned} q_i^{(t)} &= \max_{p \in \mathcal{P}_i^{(t)}} R_{i,p}^{(t)}, \\ \mathcal{S}_t &= \mathrm{Top}_{\lceil \rho N \rceil} \left\{ q_i^{(t)} \right\}_{i=1}^{N}. \end{aligned} \tag{2}$$

As shown in Eq. (2), the $\lceil \rho N \rceil$ clients with the highest communication and routing scores are selected to participate in round t. This design aligns the federated learning process with the current network condition and reduces the participation of clients whose available paths are weak, unstable, or insecure.

The selection strategy intentionally prioritizes communication reliability. However, the client participation ratio keeps the number of participating clients fixed, while the effect of non-IID data is considered in the experimental design.

3.2. Local Update Generation without Raw Data Sharing

In each communication round t, every selected IoT client receives the current global model from the Edge–Cloud server and performs local training using only its private dataset. Raw data are not transmitted to the central server, other clients, or the cloud layer. This design reduces direct raw-data exposure

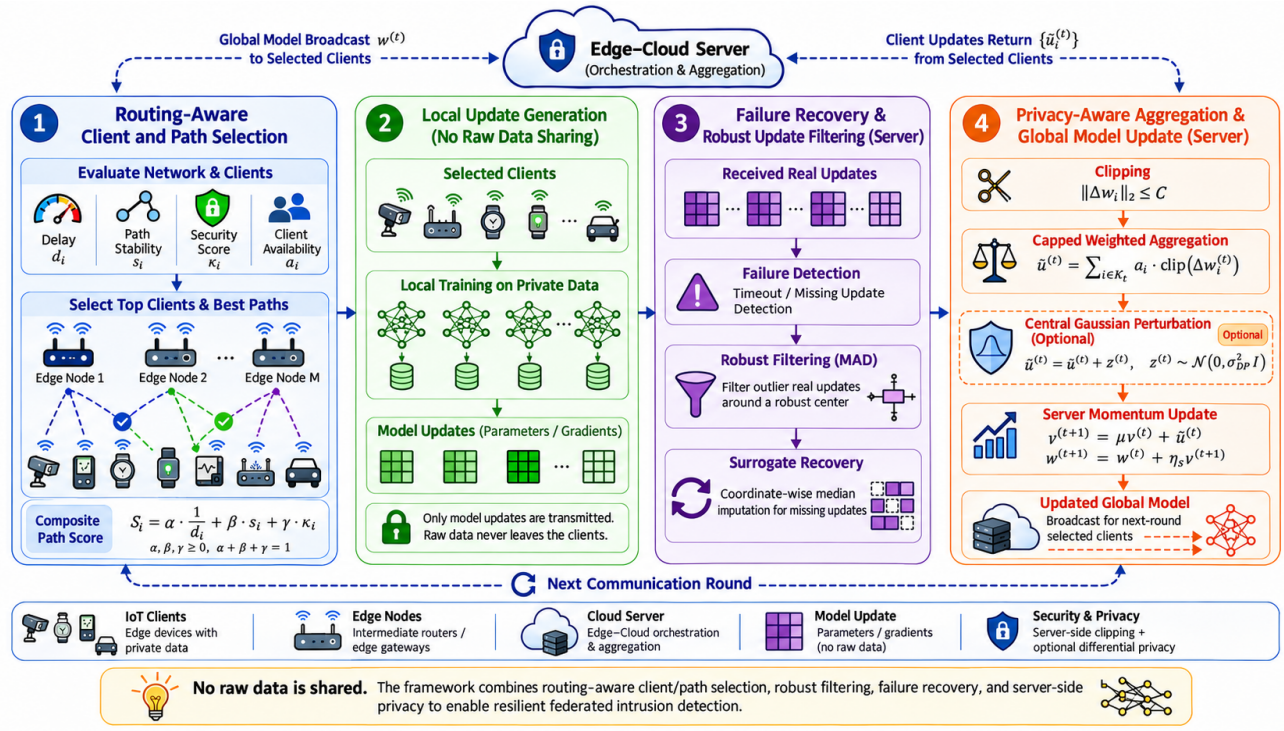


Figure 1: Overview of the routing-aware federated intrusion-detection framework. Stage 3 illustrates the real-only filtering variant, in which MAD filtering precedes surrogate recovery; the original reference configuration reverses this order. Server-side Gaussian perturbation is optional.

and communication overhead while keeping local data under the control of each IoT client.

Let $\mathcal{D}_i$ denote the private dataset of client i, containing n_i local samples. The local learning objective is defined by the empirical loss function

$$\mathcal{L}_i(w) = \frac{1}{n_i} \sum_{j=1}^{n_i} \ell\left(f(x_{ij}; w), y_{ij}\right). \tag{3}$$

In Eq. (3), $\mathcal{L}_i(w)$ denotes the empirical loss over the private dataset of client i; x_{ij} and y_{ij} represent the input and label of local sample j, respectively; $f(\cdot; w)$ is the local prediction model; and $\ell(\cdot)$ is the loss function.

Starting from the current global model $w^{(t)}$, client i performs E local epochs using learning rate η. The local optimization process and the resulting model update are defined jointly as

$$\begin{aligned} w_i^{(t,E)} &= \mathrm{LocalOpt}\left(w^{(t)}, \mathcal{D}_i, E, \eta\right), \\ \Delta w_i^{(t)} &= w_i^{(t,E)} - w^{(t)}. \end{aligned} \tag{4}$$

As expressed in Eq. (4), local training starts from the global model received at the beginning of round t. After E local epochs, client i obtains the trained local model $w_i^{(t,E)}$.

Instead of transmitting raw data or the complete local dataset, the client sends only the model difference $\Delta w_i^{(t)}$ through the selected communication path $p_i^{*(t)}$. The Edge–Cloud server subsequently processes the received updates using failure recovery, robust filtering, and privacy-aware aggregation.

3.3. *Failure Recovery and Robust Update Filtering*

In practical Edge–Cloud IoT environments, some selected clients may fail to deliver their updates during a communication round. Client failures, link outages, excessive path delay, network congestion, or adversarial behavior may cause updates to arrive late, be lost, or deviate substantially from the main update direction.

Before aggregation, the Edge–Cloud server identifies the successfully received real updates, denoted by $\mathcal{U}_t$. Two processing orders are considered. In the original reference configuration, missing-update surrogates are reconstructed before MAD filtering. In the real-only filtering variant, MAD is first applied to received real updates, and missing-update surrogates are subsequently reconstructed from the retained real updates.

For the original reference configuration, when a selected client fails to deliver its update and $|\mathcal{U}_t| \geq 2$, a surrogate update is generated using coordinate-wise median reconstruction:

$$\Delta w_{i,\mathrm{rec}}^{(t)} = \mathrm{Median}_{\mathrm{coord}}\left(\left\{\Delta w_j^{(t)} \mid j \in \mathcal{U}_t\right\}\right), \qquad |\mathcal{U}_t| \geq 2. \tag{5}$$

The recovered updates are assigned a reduced raw aggregation coefficient χ_{rec}. This lowers, but does not by itself bound, their influence, because raw weights are renormalized and capped jointly with the genuine updates (see below).

The surrogate update in Eq. (5) is not intended to reproduce the missing local update exactly. Instead, it provides a conservative replacement that reduces the disruptive effect of failed update delivery. If fewer than two valid updates are available, the unavailable client is excluded from the current aggregation

round. If no update is received, aggregation, noise injection and the privacy-accountant step are skipped and no global-model update is applied; this never occurred in the 40 controlled runs.

After recovery, the server constructs the available update set $\mathcal{V}_t$, containing both successfully received updates and recovered surrogate updates. The robust center and the deviation of each available update from this center are defined jointly as

$$\begin{aligned} m^{(t)} &= \mathrm{Median}_{\mathrm{coord}}\left(\left\{\Delta w_i^{(t)} \mid i \in \mathcal{V}_t\right\}\right), \\ d_i^{(t)} &= \left\|\Delta w_i^{(t)} - m^{(t)}\right\|_2, \qquad i \in \mathcal{V}_t. \end{aligned} \tag{6}$$

In Eq. (6), $m^{(t)}$ represents the robust central direction of the available updates, while $d_i^{(t)}$ measures the deviation of update i from this center. Large deviations may result from communication failures, highly non-IID data, poisoning attacks, or Byzantine behavior.

For compact notation, define $\mathbf{d}^{(t)} = \{d_i^{(t)}\}_{i\in\mathcal{V}_t}$. The MAD-based threshold and the resulting reliable update set are then defined as

$$\begin{aligned} \tau_t &= \mathrm{Median}\left(\mathbf{d}^{(t)}\right) + 1.4826\, k\ \mathrm{MAD}\left(\mathbf{d}^{(t)}\right), \\ \mathcal{K}_t &= \left\{i \in \mathcal{V}_t \mid d_i^{(t)} \le \tau_t\right\}. \end{aligned} \tag{7}$$

Here $\mathrm{MAD}(\mathbf{d}) = \mathrm{Median}_i\,|d_i - \mathrm{Median}(\mathbf{d})|$, and a constant 10^{-12} is added to the scaled MAD to avoid a zero-width threshold. According to Eq. (7), only updates whose deviation is less than or equal to τ_t are retained. The coefficient k controls the filtering strictness: smaller values produce stricter filtering, whereas larger values retain more diverse updates.

The filtering decision applies only to the current communication round and should not be interpreted as a permanent malicious-client label. A large deviation may also be caused by legitimate statistical heterogeneity.

If $\mathcal{K}_t$ becomes empty, the update closest to the robust center is retained as a fallback. In other words, the client with $i^* = \arg\min_{i\in\mathcal{V}_t} d_i^{(t)}$ is retained to prevent interruption of the current aggregation round.

3.4. Privacy-Aware Aggregation and Global Model Update

Following robust filtering and any configured surrogate recovery, the Edge–Cloud server constructs the aggregate from the retained updates. This stage includes update clipping, capped weighting, and server momentum. Central Gaussian perturbation is applied only when the private aggregation mode is enabled.

First, each reliable update is clipped using the clipping threshold C:

$$\overline{\Delta w_i}^{(t)} = \Delta w_i^{(t)} \min\left(1, \frac{C}{\left\|\Delta w_i^{(t)}\right\|_2 + \xi}\right), \qquad i \in \mathcal{K}_t, \tag{8}$$

where $\xi > 0$ is a small constant that prevents division by zero. Consequently, $\|\overline{\Delta w_i}^{(t)}\|_2 \le C$. Clipping limits the influence of unusually large client updates and controls the update magnitude used in the subsequent aggregation and noise-calibration procedure.

Before normalization, the raw aggregation weight of a successfully received update is proportional to its local dataset size n_i. For a reconstructed update, the corresponding raw weight is reduced by the coefficient χ_{rec}. Let $b_i^{(t)}$ denote this raw weight. The effective weight cap, the final normalized weights, and the clipped weighted aggregate are defined jointly as

$$\begin{aligned} \overline{a}_{\max}^{(t)} &= \max\left(a_{\max}, \frac{1}{|\mathcal{K}_t|}\right), \\ \left\{a_i^{(t)}\right\}_{i\in\mathcal{K}_t} &= \mathrm{CapNormalize}\left(\left\{b_i^{(t)}\right\}_{i\in\mathcal{K}_t}, \overline{a}_{\max}^{(t)}\right), \\ u^{(t)} &= \sum_{i\in\mathcal{K}_t} a_i^{(t)} \overline{\Delta w_i}^{(t)}. \end{aligned} \tag{9}$$

The resulting weights satisfy $\sum_{i\in\mathcal{K}_t} a_i^{(t)} = 1$ and $0 \le a_i^{(t)} \le \overline{a}_{\max}^{(t)}$. When fewer than $1/a_{\max} = 10$ updates are retained the cap is infeasible and the effective cap is $1/|\mathcal{K}_t|$. In the proposed pipeline (300 rounds), fewer than ten updates were retained in 92% of rounds (minimum six; largest single weight 0.167), surrogate updates jointly carried a mean weight share of 0.30 (maximum 1.0), and in 10 rounds only surrogates were retained. The weight cap therefore does not prevent surrogate updates from influencing the aggregate in the evaluated reference ordering.

For noise calibration, the implementation uses the following effective L_2-sensitivity scale for the clipped weighted aggregate:

$$\Delta_2^{(t)} = \varphi C \max_{i\in\mathcal{K}_t} a_i^{(t)}. \tag{10}$$

The coefficient φ is the adjacency-scaling factor used by the implementation. The experiments use $\varphi = 1$, corresponding to the remove/add configuration, while $\varphi = 2$ can be used for a more conservative replace-one scaling. Because the retained client set and the capped normalized weights are data-dependent, $\Delta_2^{(t)}$ is used here as an effective noise-calibration scale rather than claimed as a formal global sensitivity bound for the complete routing, filtering, and aggregation pipeline.

When the server-side Gaussian privacy mode is enabled, the weighted aggregate is perturbed as follows:

$$\begin{aligned} \widetilde{u}^{(t)} &= u^{(t)} + z^{(t)}, \\ z^{(t)} &\sim \mathcal{N}\left(0, \sigma_{\mathrm{DP}}^2 \left(\Delta_2^{(t)}\right)^2 I\right). \end{aligned} \tag{11}$$

Here, σ_{DP} is the Gaussian noise multiplier and I denotes the identity matrix. Because the Gaussian noise is added by the server after filtering, clipping, and weighted aggregation, the mechanism follows a server-side central Gaussian perturbation design and is evaluated using the estimated central-DP accounting procedure described below.

The implementation tracks the cumulative privacy budget using an RDP accountant. For accounting, the routing-aware top-k participation fraction is represented by $q = |\mathcal{S}_t|/N = 0.2$ and treated as a Poisson-sampling approximation. The noise multiplier is calibrated to the target budget. Because the actual routing-aware selection is deterministic rather than Poisson and the robust filtering stage is data-dependent, the resulting $\widehat{\varepsilon}$ is reported as an estimated central-DP budget under a trusted or

Table 2: Pseudocode of the proposed routing-aware, privacy-aware, and fault-tolerant federated learning framework.

Input: Local datasets $\{\mathcal{D}_i\}_{i=1}^{N}$; initial model $w^{(0)}$; $T, E, \eta, \eta_s, \mu, \rho, \lambda_L, \lambda_S, \lambda_Q, C, \varepsilon, \delta, k, a_{\max}, \chi_{\text{rec}}, \xi$.
Output: Final global model $w^{(T)}$ and IDS, routing, privacy, and resilience metrics.

Initialize $v^{(0)} = 0$ and calibrate σ_{DP} for the target (ε, δ).
for $t = 0, \ldots, T-1$ **do**
  Update the Edge–Cloud network state and compute candidate paths.
  Compute routing scores using latency, stability, and security; select $\mathcal{S}_t$ and assign $p_i^{*(t)}$.
  Broadcast $w^{(t)}$ to the selected clients.
  for each $i \in \mathcal{S}_t$ **in parallel do**
    Train on $\mathcal{D}_i$ for E local epochs and compute $\Delta w_i^{(t)} = w_i^{(t,E)} - w^{(t)}$.
    Transmit $\Delta w_i^{(t)}$ through $p_i^{*(t)}$.
  end for
  Detect missing, failed, or undelivered updates.
  if failures exist and $|\mathcal{U}_t| \geq 2$ **then**
    Reconstruct missing updates using the coordinate-wise median and assign reduced weight χ_{rec}.
  else
    Exclude unavailable clients from the current aggregation round.
  end if
  Construct $\mathcal{V}_t$, compute the robust center and MAD-based threshold, and obtain the reliable set $\mathcal{K}_t$.
  Clip reliable updates using C, then cap and normalize their aggregation weights.
  Compute the clipped weighted aggregate $u^{(t)}$ and its effective sensitivity scale $\Delta_2^{(t)}$.
  If central Gaussian privacy mode is enabled, add calibrated noise and update the privacy accountant; otherwise set $\widetilde{u}^{(t)} = u^{(t)}$.
  Update $v^{(t+1)} = \mu v^{(t)} + \widetilde{u}^{(t)}$ and $w^{(t+1)} = w^{(t)} + \eta_s v^{(t+1)}$.
  Compute and log IDS, routing, privacy, and resilience metrics.
end for
return $w^{(T)}$.

semi-trusted server assumption, not as a formal end-to-end guarantee. Finally, server momentum and the global-model update are performed jointly as

$$\begin{aligned} v^{(t+1)} &= \mu v^{(t)} + \widetilde{u}^{(t)}, \\ w^{(t+1)} &= w^{(t)} + \eta_s v^{(t+1)}. \end{aligned} \tag{12}$$

In Eq. (12), μ is the server momentum coefficient and η_s is the server learning rate. Momentum reduces round-to-round fluctuations caused by non-IID data, communication failures, suspicious updates, and privacy noise. The updated global model $w^{(t+1)}$ is then distributed to the clients selected for the next communication round. The complete procedure is summarized in Table 2.

4. Simulation

The proposed framework was evaluated in a simulated Edge–Cloud federated learning environment using CICIoT2023 and UNSW-NB15. The evaluation considers intrusion-detection performance, routing reliability, client participation, attack robustness, failure recovery, and privacy-related behavior. In addition to standard federated-learning baselines, the experimental design considers mechanisms motivated by modular and hierarchical resilient federated learning approaches [16, 20].

CICIoT2023 serves as the primary IoT-oriented benchmark, while UNSW-NB15 provides a complementary network-intrusion benchmark with different traffic characteristics. Component-wise comparisons are used to distinguish the effects of client selection, delivery behavior, robust filtering, surrogate recovery, weighting, and server momentum. Comparisons that modify multiple mechanisms are interpreted as pipeline-level contrasts rather than isolated ablations.

4.1. Experimental Setup and Simulation Settings

An Edge–Cloud-based federated learning environment was simulated to evaluate the proposed framework. In this environment, IoT clients do not transmit raw data to the server. Instead, only locally generated model updates are transmitted through the selected communication paths.

After receiving client updates, the Edge–Cloud server handles missing-update surrogates and MAD filtering, then performs clipping, capped weighted aggregation and momentum updating. Server-side Gaussian perturbation is optional and its accounting is diagnostic.

This experimental design enables the proposed framework to be evaluated not only in terms of intrusion detection performance, but also with respect to routing stability, communication delay, privacy awareness, and fault tolerance.

Two network-intrusion datasets were used in the experiments. CICIoT2023 was selected as the primary dataset because it contains diverse IoT traffic and multiple attack patterns and is therefore closely aligned with the Edge–Cloud IoT scenario considered in this study.

For CICIoT2023, the label of each sample was extracted from its corresponding folder or attack category. The intrusion detection task was formulated as a binary classification problem consisting of attack and benign traffic.

For the controlled experiments, CICIoT2023 was re-prepared using a file-group-disjoint split rather than the earlier row-level split. The verified prepared data contain 926,133 training and 146,626 validation records; the reserved 300,000-row test partition was not opened in this study. Within the locked validation protocol, checkpoint selection, threshold calibration, and report evaluation use separate partitions. Consequently, the reported results do not establish generalization to independent devices or captures beyond the documented file-group separation. Preparation removed feature-identical duplicates (73,867 training and 53,374 validation records) and kept 39 numerical features; file-group separation applies to the 17 of 34 classes with at least three source files, and the other 17 are row-stratified. Each validation partition contains only 1,564 benign records, so report-partition FPR is estimated from 1,564 negatives.

UNSW-NB15 was used as a complementary within-dataset benchmark with a different feature representation and traffic composition. The official training data were partitioned under the locked validation protocol, and the official test partition was not opened during these controlled experiments.

Under the locked protocol, the 175,341 official training records were divided into 150,293 training records and a 25,048-record validation pool, which was further split into calibration (8,350), selection (8,350) and report (8,348) partitions for seed 12 (partition sizes differ by at most four records across seeds); the 82,332-record official test partition was not opened. The 42 raw features expand to 194 numerical features after categorical encoding.

Network model.. Each client has three candidate paths (one per edge node). Link latency, stability, security and bandwidth are drawn uniformly at initialization (client–edge 5–80 ms, 0.75–0.99, 0.60–0.98, 20–150 Mbit/s; edge–cloud 5–50 ms, 0.85–0.995, 0.75–0.99, 100–1000 Mbit/s) and perturbed every round by clipped Gaussian noise ($\sigma = 0.025$), with client–edge outages at $p_{\text{out}} = 0.01$. Path delay is the sum of segment latencies, stability the product of segment stabilities, and security the segment minimum; the three scores are min–max normalized over the client's paths and combined with $\lambda = (0.4, 0.3, 0.3)$, using the simulator's current link state without measurement noise. An update is sent as 100 packets; first-transmission PDR is binomial with success probability equal to path stability, and update delivery is a separate Bernoulli draw with the same probability. Delay is latency plus serialization plus Gaussian jitter; queueing and retransmissions are not modelled. The full specification is in `docs/SIMULATION_SPEC.md` of the repository.

Attack model.. A malicious client sends $-\tau\,\Delta w_i^{(t)}$ with $\tau = 5.0$; exactly two of the ten selected clients are malicious in every round (exposure-controlled), independently of network state. Other attacks, collusion and adaptive attackers were not evaluated.

Training configuration.. The same base MLP architecture and common training settings were used across configurations wherever applicable. The controlled ablation comparisons use five paired random seeds (12–16). Checkpoint selection, decision-threshold calibration, and metric reporting are kept separate within the validation-based evaluation protocol. The reserved test partitions are not accessed during these ablation runs. Results distinguish selected-checkpoint and final-round performance, and error bars indicate one standard deviation across the five seeds.

4.2. Evaluation Metrics

To provide a comprehensive evaluation of the proposed framework, the evaluation metrics were grouped into four categories: intrusion-detection metrics, network-performance metrics, resilience metrics, and privacy and communication-cost metrics.

Intrusion-detection performance was evaluated using Accuracy, Precision, Recall, F1-score, ROC-AUC, PR-AUC, and the false positive rate (FPR). In all classification metrics, attack traffic was considered the positive class, whereas benign traffic was considered the negative class.

Accuracy measures the proportion of correctly classified attack and benign samples among all evaluated samples. It is defined as

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{13}$$

where TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives, respectively.

Precision measures the proportion of samples predicted as attacks that are actually attack samples:

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{14}$$

Recall, also referred to as the attack detection rate, measures the proportion of actual attack samples that are correctly detected:

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{15}$$

The F1-score is the harmonic mean of Precision and Recall and represents the balance between attack detection and false alarms:

$$\text{F1-score} = \frac{2\,\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}}. \tag{16}$$

ROC-AUC evaluates the ability of the intrusion-detection model to distinguish attack traffic from benign traffic over different decision thresholds:

$$\text{ROC-AUC} = \int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR}), \tag{17}$$

where TPR denotes the true positive rate, which is equivalent to Recall.

PR-AUC evaluates the model in the Precision–Recall space and is particularly informative when the class distribution is imbalanced:

$$\text{PR-AUC} = \int_0^1 \text{Precision}(\text{Recall})\, d(\text{Recall}). \tag{18}$$

Finally, FPR measures the proportion of benign samples that are incorrectly classified as attacks:

$$\text{FPR} = \frac{FP}{FP + TN}. \tag{19}$$

A lower FPR indicates fewer false security alerts and therefore greater operational reliability for the intrusion-detection system. The decision threshold is chosen on the calibration partition only: among 1,001 candidates with FPR ≤ 0.15 and recall ≥ 0.60, the one with the highest recall (ties: F1, balanced accuracy) is selected; if none is feasible, recall is maximized subject to FPR ≤ 0.15. Reported FPR values of 0.14–0.15 are therefore largely set by this cap and are not an operationally low false-alarm rate. Checkpoints use selection-partition ROC-AUC with a minimum improvement of 10^{-4}. Network performance

was evaluated using end-to-end delay, packet delivery ratio, path stability, and transfer efficiency.

End-to-end delay measures the average time required for a packet or model update to travel from its source to the destination. It is calculated as

$$\text{E2E-Delay} = \frac{1}{M}\sum_{m=1}^{M}\left(t_m^{\text{received}} - t_m^{\text{sent}}\right), \tag{20}$$

where M is the number of successfully delivered packets or model updates, while t_m^{sent} and t_m^{received} denote the transmission and reception times of packet or update m, respectively. A lower end-to-end delay indicates faster update transmission and more efficient routing.

First-transmission packet delivery ratio (PDR) measures the fraction of attempted packets successfully received on their initial transmission. In this simulator, successful model-update delivery is recorded separately and does not follow directly from PDR:

$$\text{PDR} = \frac{N_{\text{received}}}{N_{\text{sent}}}, \tag{21}$$

where N_{received} counts packets received on the first transmission and N_{sent} counts first attempts. The model-update delivery rate is the number of updates actually delivered divided by the number of selected update transmissions; it is reported independently of PDR in Table 4.

Path stability evaluates the proportion of selected communication paths that remain stable during the evaluated communication rounds:

$$\text{Path Stability} = \frac{N_{\text{stable}}}{N_{\text{selected}}}, \tag{22}$$

where N_{stable} is the number of stable selected paths and N_{selected} is the total number of selected paths. Each path contributes $0.5\,s + 0.5\,\varrho$ (link stability s; $\varrho = 1$ if the edge node is unchanged from the previous round), and the reported value is the mean of this score. A higher value indicates greater robustness against link-quality fluctuations, communication failures, and dynamic network changes.

Transfer efficiency measures the amount of successfully delivered data or model updates relative to the end-to-end transmission time:

$$\text{Transfer Efficiency} = \frac{D_{\text{delivered}}}{T_{\text{E2E}}}, \tag{23}$$

where $D_{\text{delivered}}$ represents the successfully delivered data or model-update volume and T_{E2E} denotes the corresponding end-to-end transmission time. A higher transfer efficiency indicates more effective use of the available network resources. Resilience was evaluated using recovery time, recovery rounds, and recovery success. Privacy and communication behavior was evaluated using the estimated privacy budget, noise-to-signal ratio, and communication cost.

Recovery time measures the time required for the federated learning system to return to a stable state after a client failure, link failure, or missing update. It is defined as

$$\text{Recovery Time} = t_{\text{recovered}} - t_{\text{failure}}. \tag{24}$$

In Eq. (24), t_{failure} denotes the time at which the failure occurs, whereas $t_{\text{recovered}}$ denotes the time at which the system returns to an acceptable stable state. A lower recovery time indicates stronger resilience and a more effective failure-recovery mechanism.

The estimated privacy budget is tracked using an RDP accountant based on the accounting participation fraction, the number of privacy-releasing rounds, the calibrated Gaussian noise multiplier, and the DP failure probability:

$$\widehat{\varepsilon} = \text{RDPAccountant}\left(q, R_{\text{DP}}, \sigma_{\text{DP}}, \delta\right). \tag{25}$$

In Eq. (25), $q = |\mathcal{S}_t|/N = 0.2$ is the participation fraction used as a Poisson-sampling approximation, $R_{\text{DP}} \le T$ is the number of rounds that actually release a noisy aggregate, σ_{DP} is the calibrated Gaussian noise multiplier, and δ is the DP failure probability. Accordingly, the reported $\widehat{\varepsilon}$ is an accounting estimate rather than a formal guarantee for the deterministic routing-aware client selection and data-dependent filtering pipeline.

The noise-to-signal ratio (NSR) measures the magnitude of the injected privacy noise relative to the magnitude of the aggregated update:

$$\text{NSR}^{(t)} = \frac{\left\|z^{(t)}\right\|_2}{\left\|u^{(t)}\right\|_2 + \xi}. \tag{26}$$

Here, $z^{(t)}$ denotes the Gaussian noise injected during round t, $u^{(t)}$ is the aggregated update before noise addition, and $\xi > 0$ prevents division by zero. A lower NSR indicates that the privacy noise has a smaller disruptive effect on the model-update direction.

Communication cost measures the total volume of model-update data transmitted by participating clients during federated training:

$$C_{\text{comm}} = \sum_{t=1}^{T}\sum_{i\in\mathcal{S}_t}\text{Size}\left(\Delta w_i^{(t)}\right). \tag{27}$$

A lower communication cost is generally preferable. However, it should be interpreted together with end-to-end delay, PDR, path stability, transfer efficiency, and intrusion-detection performance.

Recovery rounds measure the number of communication rounds required for the model or system-stability indicator to return to an acceptable range after a failure event:

$$\text{Recovery Rounds} = r_{\text{stable}} - r_{\text{failure}}. \tag{28}$$

In Eq. (28), r_{failure} denotes the communication round in which the failure occurs, whereas r_{stable} denotes the round in which stability is restored. A lower value indicates faster recovery.

The update reconstruction ratio measures the proportion of failed or missing model updates successfully reconstructed by the recovery mechanism:

$$R_{\text{upd}} = \frac{N_{\text{recovered}}}{N_{\text{failed}} + \xi}. \tag{29}$$

Here, $N_{\text{recovered}}$ denotes successfully reconstructed updates and N_{failed} denotes failed or missing updates. This ratio is not applicable when no failure occurs.

The F1 recovery fraction reported in Table 6 is a separate metric: it measures the fraction of recorded failure events for which smoothed validation F1-score returns to at least 95% of its pre-failure value. Recovery rounds measure the number of rounds required to reach that threshold.

4.3. Parameters

Table 3 summarizes the principal training, federated-learning, routing, privacy, robustness, and failure-related parameters used in the experiments.

Table 3: Main simulation and framework parameters.

Parameter	Symbol	Value
Clients / selected per round	$N/\lvert S_t\rvert$	50/10
Communication rounds / local epochs	T/E	30/2
Batch size / local learning rate	B/η	256/0.0003
Client participation ratio	ρ	0.2
Non-IID distribution	α_{Dir}	Dirichlet, 0.5
Base model	—	MLP 128−96−64, dropout 0.2
Edge nodes / link-outage probability	N_E/p_{out}	3/0.01
Routing weights	$\lambda_L/\lambda_S/\lambda_Q$	0.4/0.3/0.3
Robust filtering coefficient	k	2.5
Target privacy budgets	ε	$\approx 10,\ \approx 20$
DP failure probability	δ	10^{-5}
Calibrated Gaussian noise multiplier	σ_{DP}	0.994 ($\varepsilon \approx 10$), 0.672 ($\varepsilon \approx 20$)
Maximum aggregation weight	$a_{\max}$	0.1
Client/link failure probability	p_f	0.1
Recovered-update coefficient	χ_{rec}	0.5
Server learning rate / momentum	η_s/μ	0.5/0.9
Clipping bounds (CIC/UNSW)	C	0.05/0.10
Attack type / malicious ratio	—/r_{mal}	Sign-flip/0.2
Attack intensity	τ_{attack}	5.0
Main comparison seeds	—	12–16 (five paired seeds)
Privacy mechanism	—	Server-side Gaussian perturbation
Failure recovery	—	Coordinate-wise median

4.4. Performance Evaluation of the Proposed Framework on CICIoT2023

This section reports controlled comparisons on the CICIoT2023 validation/report partition. The original test-set-oriented Table 4 has been replaced by runs with matched seeds, partitions and attack settings. Table 4 distinguishes client selection, path assignment and aggregation/privacy settings; it reports selected-checkpoint and last-round performance separately.

4.4.1. Intrusion Detection Results on CICIoT2023

CICIoT2023 detection scores remain high across the robust controls. The non-private reference pipeline gives ROC–AUC=0.9962, whereas the proposed diagnostic $\varepsilon \approx 20$ configuration gives F1=0.9914 and ROC–AUC=0.9937. The corresponding $\varepsilon \approx 10$ variant gives ROC–AUC=0.9930. Relative to the matched non-private reference, the full $\varepsilon \approx 10$ pipeline reduces selected-checkpoint ROC–AUC by 0.0032 across five seeds. This sequential pipeline contrast includes clipping and capped weighting as well as Gaussian perturbation; no certified privacy guarantee is claimed.

The FedAvg controls illustrate why selected checkpoints must be reported alongside final-round results. With uniform selection and random paths, FedAvg achieves a selected-checkpoint ROC–AUC of 0.9934 but falls to 0.5000 at round 30 under the evaluated sign-flip attack. The matched MAD-filter control maintains final-round ROC–AUC=0.9933. The matched selected-checkpoint MAD effect is only +0.0025, so conclusions about robustness depend on the evaluation point.

4.4.2. Routing and Network Performance Results on CICIoT2023

The new controls separate two communication mechanisms. Holding uniform selection and FedAvg fixed, routing the paths rather than assigning random edges reduces mean delivered-update delay from 87.3 to 71.8 ms. First-transmission PDR increases from 0.694 to 0.782 and path stability from 0.523 to 0.855; these directions hold in all five paired seeds. Holding routed paths fixed, top-k selection further reduces delay to 66.0 ms, with PDR=0.851 and path stability=0.923. Thus, the full proposed configuration reduces delay by 24.4% compared with random-path FedAvg, but only 8.2% compared with routed-path uniform-selection FedAvg.

The packet PDR measures success of the first transmission, whereas actual model-update delivery is a separate outcome (0.851 versus 0.848 for the reference). These two quantities should not be used interchangeably. Path assignment and selection improve simulated network metrics without a clear selected-checkpoint ROC–AUC improvement. The reported transfer-efficiency values describe successful traffic per unit of delivered-update delay; network outcomes from privacy or aggregation variants with identical route policies coincide by construction.

4.5. Complementary Benchmark Evaluation on UNSW-NB15

UNSW-NB15 is a second within-dataset benchmark with distinct traffic and feature representations; it does not constitute cross-dataset transfer. The primary evidence in Table 4 comes from the same locked validation-only protocol and five paired seeds used on CICIoT2023. The compared arms are executed FedAvg, MAD-filter and DP+FedAvg configurations with explicit path/selection policies, plus the non-private and Gaussian reference pipelines. The previously mentioned "Modular Edge FL-inspired" and "MCHFL-style" methods were not executed and are not presented as evaluated baselines.

4.5.1. Intrusion Detection Results on UNSW-NB15

On the report partition, the proposed diagnostic $\varepsilon \approx 20$ configuration achieves F1=0.9065, ROC–AUC=0.9522 and FPR=0.1455. FedAvg with uniform selection and random paths gives F1=0.7094, ROC–AUC=0.8759 and FPR=0.1449; this multi-component contrast must not be attributed to routing. In the matched random-path, uniform-selection comparison, enabling MAD filtering improves ROC–AUC by 0.0978 (five-seed paired 95% interval 0.017–0.179). By contrast, switching path assignment while holding the aggregation pipeline fixed does not reveal a clear detection-AUC effect. The non-private reference achieves ROC–AUC=0.9838; the matched proposed Gaussian pipeline at diagnostic $\varepsilon \approx 10$ achieves 0.9419, a paired change of −0.0419.

Table 4: Controlled comparison on the locked validation/report partition (five paired seeds, 12–16; mean (SD)). AUC_{last} is last-round ROC–AUC; Delay is in ms for delivered updates, PDR is first-transmission packet success, Deliv. is update-delivery success, Stab. is path stability, and TE is transfer efficiency in Mbit/s. TK = top-k selection, UR = uniform selection, R = routed paths, and X = random paths. The ε labels refer to diagnostic RDP targets, not certified guarantees.

Configuration	F1	AUC	AUC_{last}	FPR	Delay	PDR	Deliv.	Stab.	TE
CICIoT2023									
FedAvg (TK/R)	0.9794 (0.0259)	0.9813 (0.0211)	0.4625 (0.0843)	0.1513 (0.0122)	66.0 (5.9)	0.851 (0.015)	0.848 (0.036)	0.923 (0.007)	12.9 (1.3)
FedAvg (UR/R)	0.9904 (0.0010)	0.9930 (0.0015)	0.5516 (0.1154)	0.1400 (0.0108)	71.8 (4.2)	0.782 (0.032)	0.779 (0.022)	0.855 (0.023)	12.1 (0.9)
FedAvg (UR/X; E1)	0.9909 (0.0011)	0.9934 (0.0013)	0.5000 (0.0000)	0.1440 (0.0079)	87.3 (5.2)	0.694 (0.035)	0.685 (0.038)	0.523 (0.010)	9.9 (0.8)
MAD filter (UR/R)	0.9933 (0.0003)	0.9962 (0.0003)	0.9958 (0.0007)	0.1473 (0.0122)	71.8 (4.2)	0.782 (0.032)	0.779 (0.022)	0.855 (0.023)	12.1 (0.9)
MAD filter (UR/X; E2)	0.9932 (0.0003)	0.9960 (0.0003)	0.9933 (0.0050)	0.1477 (0.0146)	87.3 (5.2)	0.694 (0.035)	0.685 (0.038)	0.523 (0.010)	9.9 (0.8)
DP+FedAvg (UR/X; E3)	0.9827 (0.0092)	0.9738 (0.0186)	0.9732 (0.0181)	0.1481 (0.0213)	87.3 (5.2)	0.694 (0.035)	0.685 (0.038)	0.523 (0.010)	9.9 (0.8)
Ref. no noise (TK/R)	0.9934 (0.0002)	0.9962 (0.0003)	0.9960 (0.0006)	0.1441 (0.0181)	66.0 (5.9)	0.851 (0.015)	0.848 (0.036)	0.923 (0.007)	12.9 (1.3)
Proposed $\varepsilon \approx 10$	0.9911 (0.0004)	0.9930 (0.0014)	0.9931 (0.0016)	0.1407 (0.0184)	66.0 (5.9)	0.851 (0.015)	0.848 (0.036)	0.923 (0.007)	12.9 (1.3)
Proposed $\varepsilon \approx 20$ (E4)	0.9914 (0.0011)	0.9937 (0.0013)	0.9938 (0.0013)	0.1468 (0.0124)	66.0 (5.9)	0.851 (0.015)	0.848 (0.036)	0.923 (0.007)	12.9 (1.3)
UNSW-NB15									
FedAvg (TK/R)	0.7703 (0.2023)	0.8870 (0.0856)	0.6043 (0.1430)	0.1425 (0.0149)	76.8 (6.3)	0.851 (0.015)	0.848 (0.036)	0.923 (0.007)	20.4 (1.8)
FedAvg (UR/R)	0.7217 (0.2069)	0.8752 (0.0863)	0.6493 (0.1749)	0.1464 (0.0158)	82.0 (4.8)	0.782 (0.032)	0.779 (0.022)	0.855 (0.023)	19.2 (1.3)
FedAvg (UR/X; E1)	0.7094 (0.2163)	0.8759 (0.0839)	0.5551 (0.1230)	0.1449 (0.0168)	97.4 (5.7)	0.694 (0.035)	0.685 (0.038)	0.523 (0.010)	16.1 (1.2)
MAD filter (UR/R)	0.9526 (0.0026)	0.9825 (0.0018)	0.9770 (0.0083)	0.1431 (0.0143)	82.0 (4.8)	0.782 (0.032)	0.779 (0.022)	0.855 (0.023)	19.2 (1.3)
MAD filter (UR/X; E2)	0.9425 (0.0211)	0.9737 (0.0208)	0.8334 (0.2371)	0.1422 (0.0168)	97.4 (5.7)	0.694 (0.035)	0.685 (0.038)	0.523 (0.010)	16.1 (1.2)
DP+FedAvg (UR/X; E3)	0.8314 (0.0480)	0.8854 (0.0221)	0.7894 (0.1357)	0.1534 (0.0063)	97.4 (5.7)	0.694 (0.035)	0.685 (0.038)	0.523 (0.010)	16.1 (1.2)
Ref. no noise (TK/R)	0.9544 (0.0021)	0.9838 (0.0029)	0.9826 (0.0045)	0.1449 (0.0085)	76.8 (6.3)	0.851 (0.015)	0.848 (0.036)	0.923 (0.007)	20.4 (1.8)
Proposed $\varepsilon \approx 10$	0.9005 (0.0335)	0.9419 (0.0166)	0.9285 (0.0187)	0.1488 (0.0102)	76.8 (6.3)	0.851 (0.015)	0.848 (0.036)	0.923 (0.007)	20.4 (1.8)
Proposed $\varepsilon \approx 20$ (E4)	0.9065 (0.0351)	0.9522 (0.0149)	0.9517 (0.0149)	0.1455 (0.0067)	76.8 (6.3)	0.851 (0.015)	0.848 (0.036)	0.923 (0.007)	20.4 (1.8)

The reported FPR is calibrated on validation; performance and false-alarm rates on the untouched official test partition remain unmeasured under this protocol.

4.5.2. *Adversarial Robustness under Sign-flip Poisoning*

The experiments examine adversarial updates and robust aggregation under a 20% sign-flip attack on UNSW-NB15. Figure 2 distinguishes selected-checkpoint ROC-AUC from final-round ROC-AUC and reports MAD-filtering outcomes across different numbers of genuinely received updates.

4.5.3. *Routing and Network Performance Results on UNSW-NB15*

The network-level results for UNSW-NB15 are summarized in Table 4, allowing comparison with the corresponding metrics reported for CICIoT2023. Figure 3 compares routing reliability, client participation, and detection performance under alternative selection and delivery configurations. The comparisons are descriptive when multiple configuration settings differ.

The six panels in Figure 3 illustrate why communication reliability and client-selection fairness must be considered together. The top-k reference favors routes with strong network scores but concentrates participation among fewer clients; coverage-aware selection provides a different balance. The packet- coupled configuration also changes the delivery process, so its performance should be interpreted as a pipeline-level comparison.

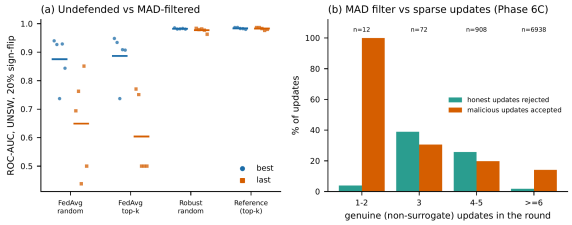


Figure 2: Sign-flip attack (20%) on UNSW-NB15: (a) selected-checkpoint and final-round ROC-AUC; (b) MAD decisions by the number of genuine received updates.

The method matrix separates routing from aggregation. The uniform-selection FedAvg and MAD-filter controls with random paths are compared against the corresponding routed-path controls, with selected clients, attack assignments and failure draws matched. These contrasts indicate improvements in communication measures, not an independently detected accuracy gain. In particular, on UNSW-NB15 routed uniform-selection FedAvg reduces delivered-update delay from 97.4 to 82.0 ms, increases first-transmission PDR from 0.694 to 0.782 and increases path stability from 0.523 to 0.855. The top-k routed configuration decreases delay further to 76.8 ms. Five-seed uncertainty and the use of independently drawn packet-delivery uniforms between path policies limit inference.

Figure 4 compares communication outcomes under independent and coupled network settings. Update delivery, first-transmission packet success, and delivery delay are reported separately.

First-transmission PDR and successful update delivery are distinct outcomes. Figure 4 reports both with latency and individual-seed variation. Because coupling affects multiple communication processes, this comparison is descriptive rather than an isolated estimate of one mechanism's effect.

For UNSW-NB15, the selected-checkpoint advantage over FedAvg is associated primarily with robust aggregation under the controlled comparison, whereas routed-path assignment and top-k selection improve network outcomes. The proposed $\varepsilon \approx 20$ configuration has delivered-update delay 76.8 ms and first-transmission PDR=0.851, compared with 97.4 ms and 0.694 for random-path uniform-selection FedAvg. This comparison changes both path assignment and selection, so its combined effect is not attributable to either factor alone. Results are exploratory: with only five paired seeds, no two-sided exact sign-flip test can reach $p < 0.05$. Detection results are validation/report partition outcomes rather than independent test-set results.

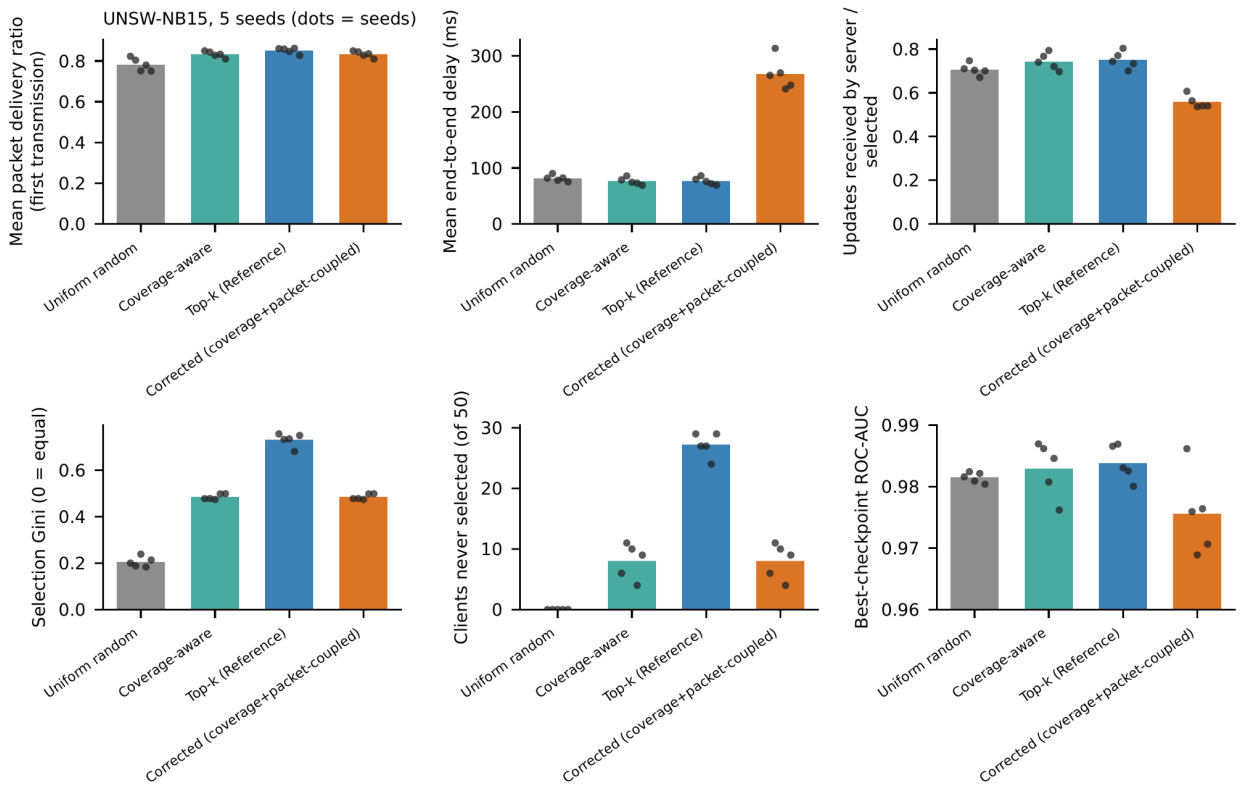


Figure 3: UNSW-NB15 routing, participation fairness, and detection over five seeds: (a) first-transmission PDR, (b) delay, (c) received-update ratio, (d) selection Gini, (e) never-selected clients, and (f) selected-checkpoint ROC-AUC.

4.6. Client-Selection Concentration and Exposure

Client-selection policies differ in how participation is distributed across clients. Figure 5 shows the cumulative share of selections across clients ranked by selection frequency. Steeper curves indicate greater concentration among a smaller subset of clients.

The selection curves complement the Gini and never-selected-client statistics in Figure 3. The top-k policy shows more concentrated participation, while uniform selection spreads opportunities across more clients. Coverage-aware and history-based policies occupy different positions between these extremes. Hence, high delivery reliability need not imply balanced participation across the client population.

Figure 6 examines the association between malicious-client exposure and last-round detection performance. The results show variation across network settings and random seeds; the relationship is descriptive rather than causal.

The individual-seed points in Figure 6 show that similar

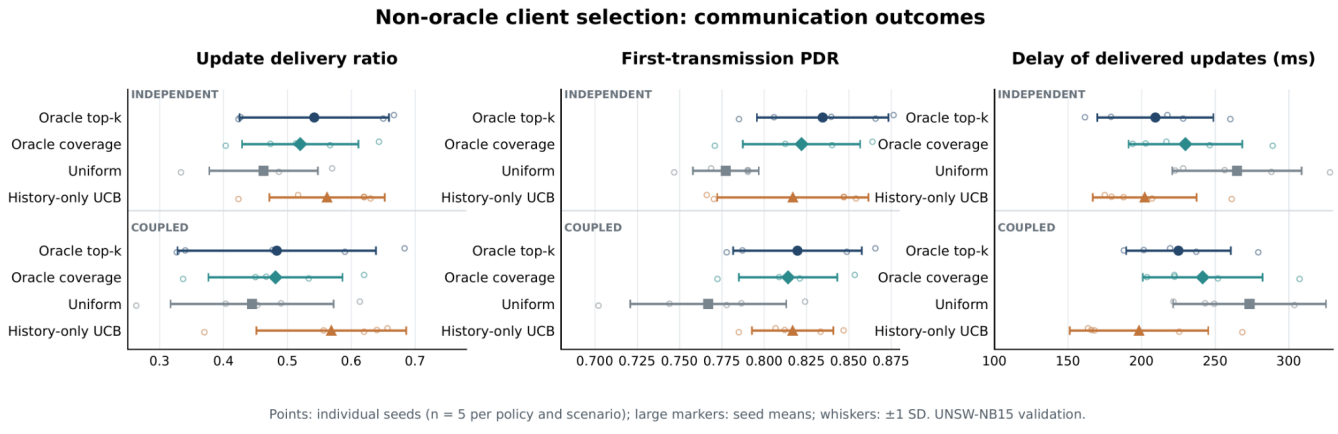


Figure 4: Communication outcomes on UNSW-NB15: (a) update delivery ratio, (b) first-transmission PDR, and (c) delay of delivered updates. Markers and error bars summarize five seeds.

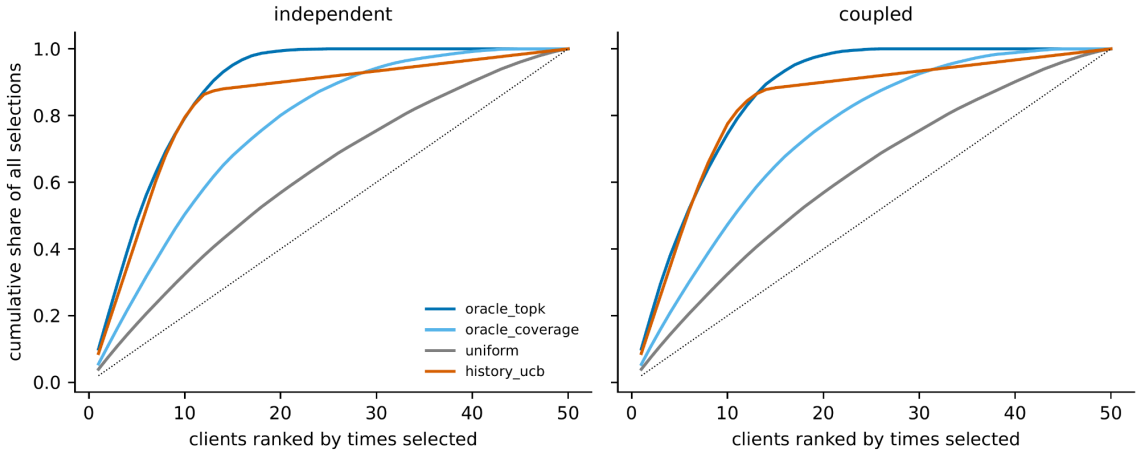


Figure 5: Cumulative client-selection concentration in independent and coupled networks across four selection policies. Curves represent five-seed means.

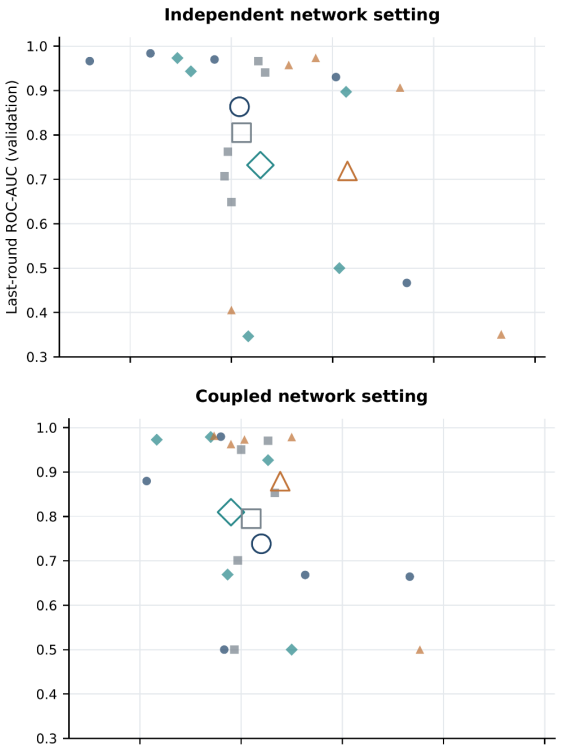


Figure 6: Malicious-client exposure versus final validation ROC-AUC under independent and coupled settings. Circle, diamond, square, and triangle denote top-k, coverage, uniform, and UCB policies, respectively; hollow markers show policy means ($n = 5$).

malicious-client exposure can coincide with different final-round detection outcomes. The hollow markers summarize policy-level means, but they do not remove within-policy variability. This descriptive relationship should not be taken as evidence that exposure alone determines ROC-AUC; routing, aggregation, and stochastic training conditions also differ.

4.7. Ablation

We examined the contribution of the framework components on CICIoT2023 and UNSW-NB15 through paired comparisons using five random seeds (12–16). The evidence comprises 130 audited historical runs and 40 current-code runs. The latter reproduced the reference configuration and evaluated the robust-filtering and privacy-pipeline controls; the other historical variants were verified against the evaluation ledger rather than fully rerun. All reported ROC–AUC values in this section refer to the validation/report partition, not the reserved test set. We distinguish the selected validation checkpoint from the last communication round (round 30). Differences are computed for matched seeds as variant minus comparator. Single-factor controls are interpreted separately from comparisons that change multiple stages.

4.7.1. Effect of Robust Filtering on Training Stability

Removing median absolute deviation (MAD)-based filtering caused the largest observed single-factor reduction in last-round ROC–AUC. As shown in Figure 7, the mean paired selected-checkpoint differences were −0.011 on CICIoT2023 and −0.093 on UNSW-NB15, with reported uncertainty intervals spanning zero. In contrast, last-round differences reached −0.309 and −0.518, respectively, and all five seeds showed lower final-round ROC–AUC without robust filtering. The last-round ROC–AUC fell more than 0.05 below the selected checkpoint for four of five unfiltered CICIoT2023 runs and all five unfiltered UNSW-NB15 runs; this did not occur in any matched reference run. Thus, checkpoint selection can conceal substantial late-round instability when anomalous updates are not filtered.

4.7.2. Other Components and Privacy-Pipeline Effects

Figure 8 compares paired ROC–AUC changes for the remaining controls. Removing surrogate recovery, disabling server momentum while holding the server learning rate at $\eta_s = 0.5$, and applying MAD filtering to genuine updates before surrogate reconstruction did not produce clearly distinguishable selected-checkpoint effects across the five seeds. This is not evidence of

Table 5: Audited mean paired ROC–AUC differences over five seeds. Filtering is compared with the matched reference; privacy effects are ordered contrasts, with Gaussian noise added after clipping and capped weighting.

Contrast (evaluation point)	CICIoT2023	UNSW-NB15
No MAD filter (selected)	−0.011	−0.093
No MAD filter (last)	−0.309	−0.518
Clipping + cap (selected)	−0.0017	−0.0262
Added Gaussian (selected)	−0.0015	−0.0157
Full privacy pipeline (selected)	−0.0032	−0.0419

equivalence. Client-selection policies likewise showed small and variable detection differences, although participation became more balanced: the selection Gini coefficient changed from approximately 0.73 for top-k selection to 0.49 for coverage-aware selection and 0.21 for uniform selection; the mean numbers of never-selected clients were 27.2, 8, and 0, respectively.

The ordered privacy comparison in Table 5 separates the effect of clipping and capped weighting without Gaussian noise from the incremental effect of adding Gaussian perturbation to that same pipeline. At the selected checkpoint, clipping and capped weighting yielded differences of −0.0017 and −0.0262 on CICIoT2023 and UNSW-NB15. Subsequent Gaussian perturbation contributed further differences of −0.0015 and −0.0157, giving full-pipeline contrasts of −0.0032 and −0.0419, respectively. The non-noise stages therefore represent approximately 53% and 63% of these point-estimate losses. These are conditional, sequential comparisons rather than a factorial attribution of independent effects; interactions were not separately estimated. The associated privacy budget ($\widehat{\varepsilon} \approx 10$) is an accounting estimate, not a certified end-to-end guarantee.

Communication changes require similar care. On CICIoT2023, packet-coupled delivery reduced the delivered-update ratio from approximately 0.848 to 0.681, increased delay from roughly 66–77 ms to 250–263 ms, and reduced genuine retained updates from about 5.9 to 4.3 per round. Because this configuration changes the delivery process, it is not an isolated ablation of the routing algorithm. Likewise, a historical configuration combining zero momentum with server learning rate 5.0 cannot identify the effect of momentum alone. With only five paired seeds, a

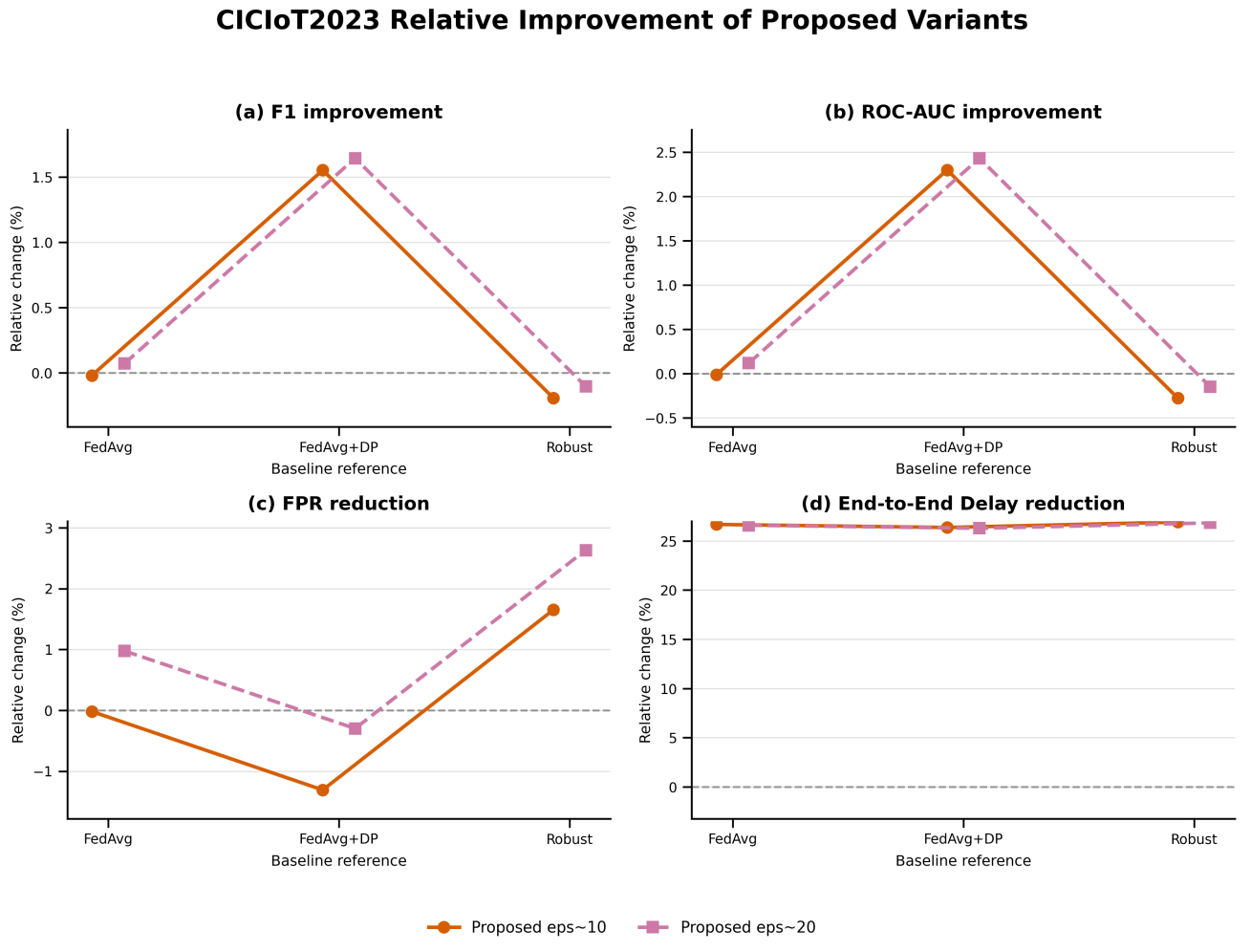


Figure 7: Effect of disabling MAD-based robust filtering on validation/report-partition ROC–AUC: (a) CICIoT2023 and (b) UNSW-NB15. Each pair of markers represents the same random seed ($n = 5$); horizontal black bars indicate five-seed means. Selected-checkpoint results are distinguished from round-30 results.

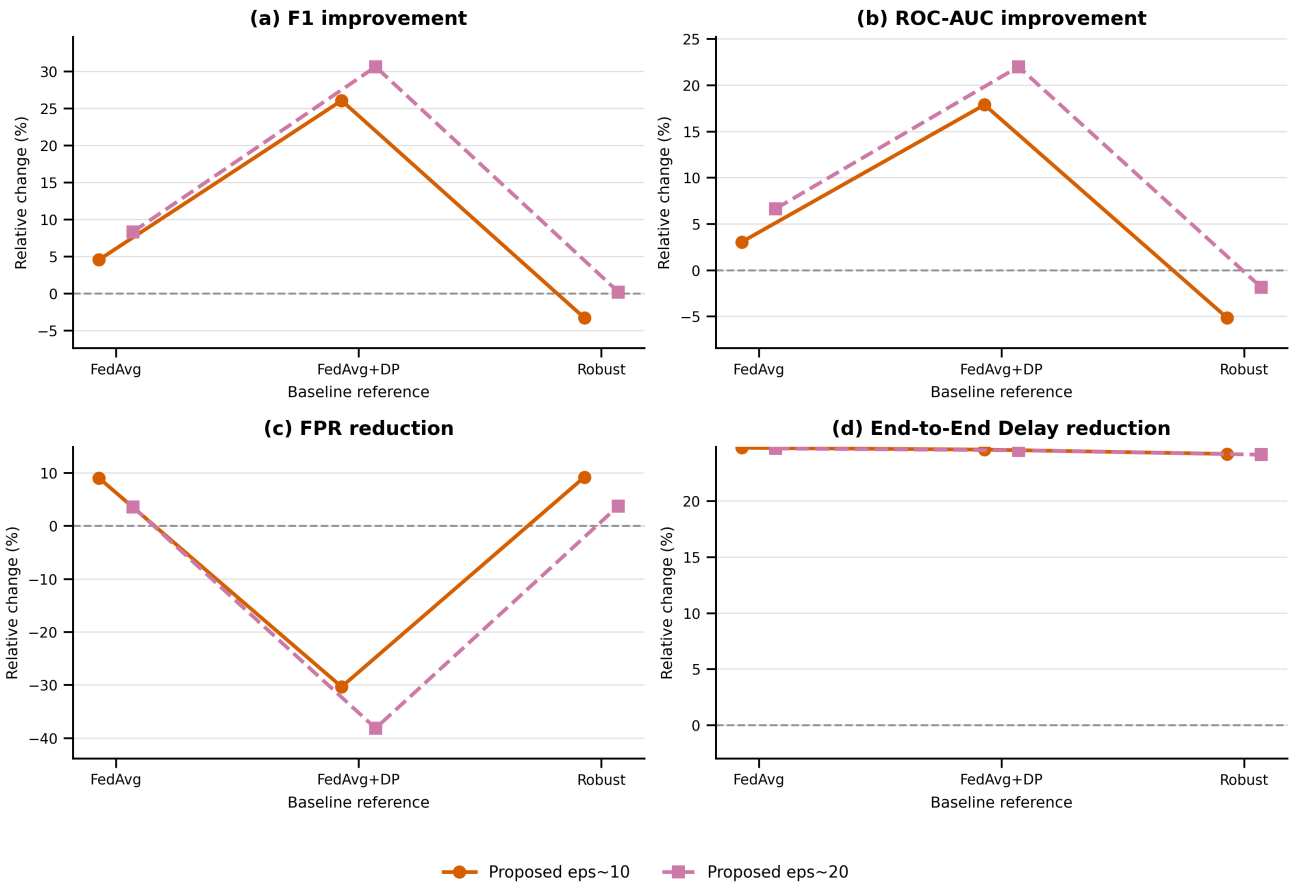


Figure 8: Paired ROC–AUC changes (variant minus matched comparator) for CICIoT2023 and UNSW-NB15. Blue circles denote selected checkpoints and orange diamonds denote the last round; filled and hollow markers distinguish single-factor and pipeline-level contrasts. Horizontal intervals summarize uncertainty across paired seeds. The horizontal axis uses a symmetric-logarithmic scale, and the privacy contrasts have different matched comparators, as specified in the text.

two-sided exact sign-flip test has a minimum attainable p-value of 0.0625; small observed differences should therefore not be interpreted as statistical equivalence or definitive component effects. The reported comparisons use audited metrics and exclude unregistered checkpoint files.

4.8. Controlled Failure Sensitivity and Privacy Diagnostics

The audited reference configuration was examined under explicit client-failure probabilities of 0, 0.1, and 0.3 on UNSW-NB15, using five random seeds. These experiments represent a reference-configuration sensitivity analysis, not an isolated evaluation of the full privacy-enabled pipeline. Injected client failures occur in addition to baseline network non-delivery; therefore, a zero injected failure probability does not imply a failure-free network.

4.8.1. Failure Sensitivity

Table 6 summarizes the observed failure behavior. Mean failed clients per round increased from 1.52 to 4.11 across the evaluated settings, while the validation-F1 recovery fraction remained between 0.962 and 0.973.

Table 6: Audited failure sensitivity of the UNSW-NB15 reference configuration (five-seed means).

p_f	Failures per round	F1 recovery fraction	Recovery rounds
0.0	1.52	0.962	1.12
0.1	2.50	0.963	1.11
0.3	4.11	0.973	1.20

The reported recovery fraction measures the proportion of failure events followed by restoration of smoothed validation F1-score to at least 95% of its pre-failure value. It is distinct from the proportion of missing client updates successfully reconstructed. Recovery success did not show a monotonic decline as injected failures increased.

The observed packet delivery ratio and mean delay remained approximately 0.8511 and 76.65 ms across all three settings.

Since explicit client failures did not modify the network-delivery process, these constant network metrics do not establish an independent failure-mitigation effect of routing-aware selection.

4.8.2. Privacy Diagnostics

For the audited Gaussian reference configuration, the estimated privacy-accounting budget was approximately $\widehat{\varepsilon} = 9.999$. The corresponding noise-to-signal ratios were approximately 23.53 on CICIoT2023 and 35.29 on UNSW-NB15.

The new Table 4 also reports completed DP+FedAvg and $\varepsilon \approx 20$ controls. Comparisons across these configurations are multi-factor and should not be used to attribute utility effects solely to Gaussian noise. Moreover, deterministic client selection and data-dependent filtering are not fully represented by the accounting approximation. Thus, the reported privacy budget is not a certified end-to-end differential-privacy guarantee.

The ablation results in Section 4.7 also indicate that clipping and capped weighting contribute to the observed utility loss alongside Gaussian noise. Accordingly, privacy-related utility changes must be interpreted at the pipeline level.

5. Conclusion

The controlled evaluation examined routing-aware update delivery, median-based surrogate imputation, MAD filtering and optional Gaussian perturbation in simulated Edge–Cloud intrusion detection. In CICIoT2023, the proposed diagnostic $\varepsilon \approx 20$ configuration achieves F1=0.9914 and ROC–AUC=0.9937 on the validation/report partition. Mean delivered-update delay is 66.0 ms, compared with 87.3 ms for uniform-selection FedAvg using random paths and 71.8 ms for the corresponding routed-path baseline. On UNSW-NB15, the proposed configuration achieves F1=0.9065 and ROC–AUC=0.9522. The matched MAD-filter control improves ROC–AUC by approximately 0.098 over FedAvg with other conditions held fixed. Path assignment and client selection improve simulated network metrics without an independently detectable gain in selected-checkpoint detection AUC.

The separate five-seed ablation demonstrates the importance of MAD filtering for last-round stability, while the privacy pipeline reduces selected-checkpoint AUC relative to the non-private reference. These estimates have limited resolution with only five paired seeds. Results are confined to a simulated network, a sign-flip attacker model and validation/report partitions; they do not establish independent official-test performance or transfer to deployed networks. Accounting values $\widehat{\varepsilon} \approx 10$ and 20 are diagnostics rather than certified end-to-end differential-privacy guarantees. Future studies should validate test-set transfer under a frozen protocol, assess operational false-alarm requirements and evaluate network realism, adversarial adaptation and privacy calibration.

Code Availability

The source code, configurations, run manifest and analysis scripts are publicly available at github.com/RezaAzarnia/routing-aware-fl-edgecloud-iot-ids. A versioned snapshot (release `v1.0.4`) is archived on Zenodo, 10.5281/zenodo.24031823. It contains the code, the locked configurations and the Table 4 controlled-study manifest and analysis scripts.

Data Availability

The public CICIoT2023 and UNSW-NB15 datasets were used; no new dataset was generated. The raw datasets are not redistributed. The Zenodo archive 10.5281/zenodo.24031823 includes the locked experiment manifest and the published per-seed, summary and paired-effect tables of the controlled five-seed study (seeds 12–16). The raw per-run output directories (round logs, validation predictions, model checkpoints) are not published and are therefore not archived; values that depend on them can be checked only through the published tables and the manifest.

References

[1] I. A. Soomro, et al., SecureDyn-FL: A robust privacy-preserving federated learning framework for intrusion detection in IoT networks, IEEE Transactions on Network and Service Management 23 (2026) 1742–1765.

[2] R. Carillo, et al., A federated and incremental network intrusion detection system for IoT emerging threats, IEEE Transactions on Network and Service Management 23 (2026) 3865–3880.

[3] S. Latif, D. Djenouri, Adaptive multi-model hierarchical federated learning for robust IoT intrusion detection, Sensors 26 (10) (2026) 3198.

[4] J. Yang, Y. Chen, S2D-FL: Sparsity-guided and trust-aware federated learning for joint robustness and privacy in IoT edge computing, IEEE Open Journal of the Computer Society 7 (2026) 914–923. doi:10.1109/OJCS.2026.3689594.

[5] J. Shu, et al., A trustworthy federated learning framework for joint privacy and byzantine resilience, IEEE Internet of Things Journal 13 (7) (2026) 12673–12686.

[6] W. Liu, et al., Anonymous and byzantine-robust federated learning with secure and efficient aggregation, IEEE Transactions on Dependable and Secure Computing 23 (3) (2026) 5214–5225.

[7] G. Zhu, et al., BPFLH: Byzantine-robust privacy-preserving federated learning for heterogeneous data, IEEE Transactions on Dependable and Secure Computing 23 (3) (2026) 6103–6118.

[8] M. Zhang, et al., EndPCA: Ensemble defense with provably convergent aggregation against poisoning attacks in federated learning, IEEE Transactions on Information Forensics and Security 21 (2026) 1438–1453.

[9] Y. Zhang, et al., "Malicious or Benign?": Enhancing the contribution of model updates in byzantine-robust heterogeneous federated learning, IEEE Transactions on Network Science and Engineering 13 (2026) 6027–6040.

[10] S. H. Shah, I. Walker, M. Borowczak, TriSAFE: Transcript-bound verifiable secure aggregation with differential privacy and timing defenses for gateway-assisted IoT federated learning, IEEE Access 14 (2026) 89354–89379.

[11] L. Wang, et al., DeSA: Decentralized secure aggregation for federated learning in zero-trust device-to-device networks, IEEE Transactions on Information Forensics and Security 21 (2026) 1987–2001.

[12] Y. Tang, et al., FedCST: Reliable federated learning under clustered data distribution in industrial internet of things, Computer Networks 280 (2026) 112209.

[13] W. Abdallah, Federated edge intelligence for secure and adaptive routing in IoT: A GRU-RL-based framework, Journal of Combinatorial Optimization 51 (2) (2026) 1–28.

[14] Y. Zhang, et al., Adaptive routing protocol for large-scale power internet of things based on edge computing and federated learning, Scientific Reports 16 (2026) 12246.

[15] A. G. Alanazi, H. A. Alanazi, EdgeGuard-AI: Zero-trust and load-aware federated scheduling for secure and low-latency IoT edge networks, Sensors 26 (6) (2026) 1989.

[16] L. Almeida, et al., Federated learning for a dynamic edge: A modular and resilient approach, Sensors 25 (12) (2025) 3812. doi:10.3390/s25123812.

[17] S. R. S, R. Aburukba, Federated learning-driven IoT request scheduling for fault tolerance in cloud data centers, Mathematics 13 (13) (2025) 2198. doi:10.3390/math13132198.

[18] N. Albanbay, et al., Federated learning-based intrusion detection in IoT networks: Performance evaluation and data scaling study, Journal of Sensor and Actuator Networks 14 (4) (2025) 78. doi:10.3390/jsan14040078.

[19] T. Q. Al-Ghadi, et al., Leveraging federated learning for DoS attack detection in IoT networks based on ensemble feature selection and deep learning models, Cyber Security and Applications 3 (2025) 100098. doi:10.1016/j.csa.2025.100098.

[20] X. Chen, et al., Multicenter hierarchical federated learning with fault-tolerance mechanisms for resilient edge computing networks, IEEE Transactions on Neural Networks and Learning Systems 36 (1) (2025) 47–61.

[21] T. Geng, J. Liu, C.-T. Huang, A privacy-preserving federated learning framework for IoT environment based on secure multi-party computation, in: 2024 IEEE Annual Congress on Artificial Intelligence of Things (AIoT), IEEE, 2024, pp. 117–122. doi:10.1109/AIoT63253.2024.00032.

[22] M. Bhardwaj, et al., A novel federated learning approach for routing optimisation in opportunistic IoT networks, International Journal of Sensor Networks 46 (1) (2024) 24–38.

[23] R. Haripriya, et al., Optimizing fault tolerance and latency of federated learning using edge servers and pre-trained model, IEEE Access 13 (2025) 158737–158750.

[24] Y. Liu, et al., Secure federated learning with configurable reliability in heterogeneous edge computing, Computer Networks 274 (2026) 111808. doi:10.1016/j.comnet.2025.111808.

[25] F. R. Mughal, et al., Adaptive federated learning for resource-constrained IoT devices through edge intelligence and multi-edge clustering, Scientific Reports 14 (1) (2024) 28746.

[26] X. Liu, X. Dong, N. Jia, W. Zhao, Federated learning-oriented edge computing framework for the IIoT, Sensors 24 (13) (2024) 4182. doi:10.3390/s24134182.

[27] J. Zhou, S. Pal, C. Dong, K. Wang, Enhancing quality of service through federated learning in edge–cloud architecture, Ad Hoc Networks 156 (2024) 103430. doi:10.1016/j.adhoc.2024.103430.

[28] Y. Lan, et al., An efficient and secure adaptive federated learning method based on CKKS for data processing in the internet of things, Internet of Things 33 (2025) 101725. doi:10.1016/j.iot.2025.101725.

[29] H. Peng, C. Wu, Y. Xiao, FD-IDS: Federated learning with knowledge distillation for intrusion detection in Non-IID IoT environments, Sensors 25 (14) (2025) 4309. doi:10.3390/s25144309.

[30] L. Wang, Y. Li, L. Zuo, Trust management for IoT devices based on federated learning and blockchain, The Journal of Supercomputing 81 (2025) 232. doi:10.1007/s11227-024-06715-4.

[31] M. Rahmati, A. Pagano, Federated learning-driven cybersecurity framework for IoT networks with privacy preserving and real-time threat detection capabilities, Informatics 12 (3) (2025) 62. doi:10.3390/informatics12030062.

[32] D. Li, et al., Ubiquitous intelligent federated learning privacy-preserving scheme under edge computing, Future Generation Computer Systems 144 (2023) 205–218.

[33] M. Bayat, et al., Enhancing secure IoT data sharing through dynamic Q-learning and blockchain at the edge, Scientific Reports 15 (1) (2025) 39153.

[34] N. N. Albogami, Intelligent deep federated learning model for enhancing security in internet of things enabled edge computing environment, Scientific Reports 15 (1) (2025) 4041.

[35] V. Padmavathi, R. Saminathan, A federated edge intelligence framework with trust-based access control for secure and privacy-preserving IoT systems, Scientific Reports 15 (1) (2025) 35832.